



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

Lập trình Hướng Đối tượng Object Oriented Programming

\$4 Các kỹ thuật xây dựng lớp





Nội dung

- Đóng gói
- Hàm khởi tạo
- Nạp chồng
- Kết tập





ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

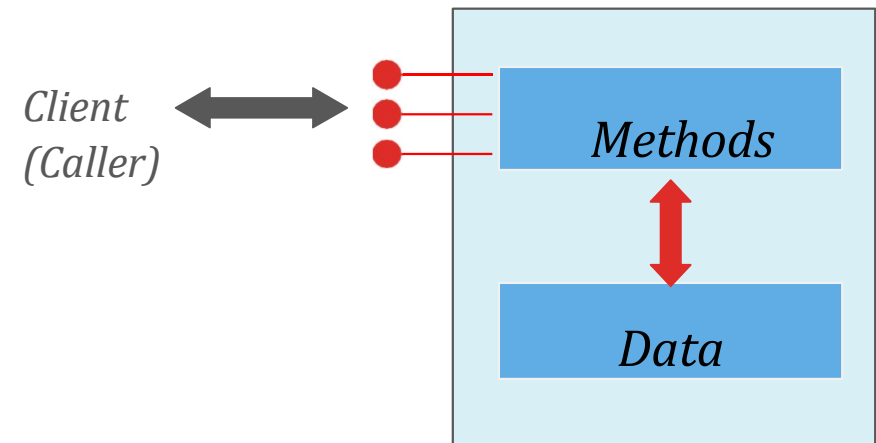
KHOA TOÁN – TIN

1. Đóng gói / encapsulation



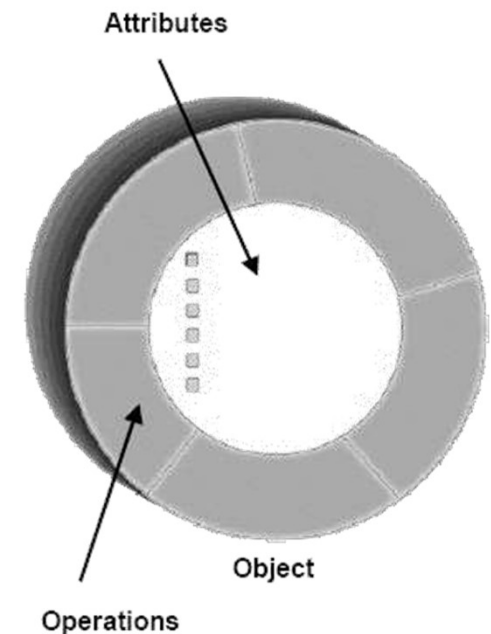
Đóng gói

- Một đối tượng có hai khung nhìn:
 - Bên trong: Chi tiết về các thuộc tính và các phương thức của lớp tương ứng với đối tượng
 - Bên ngoài: Các dịch vụ mà một đối tượng có thể cung cấp và cách đối tượng đó tương tác với phần còn lại của hệ thống



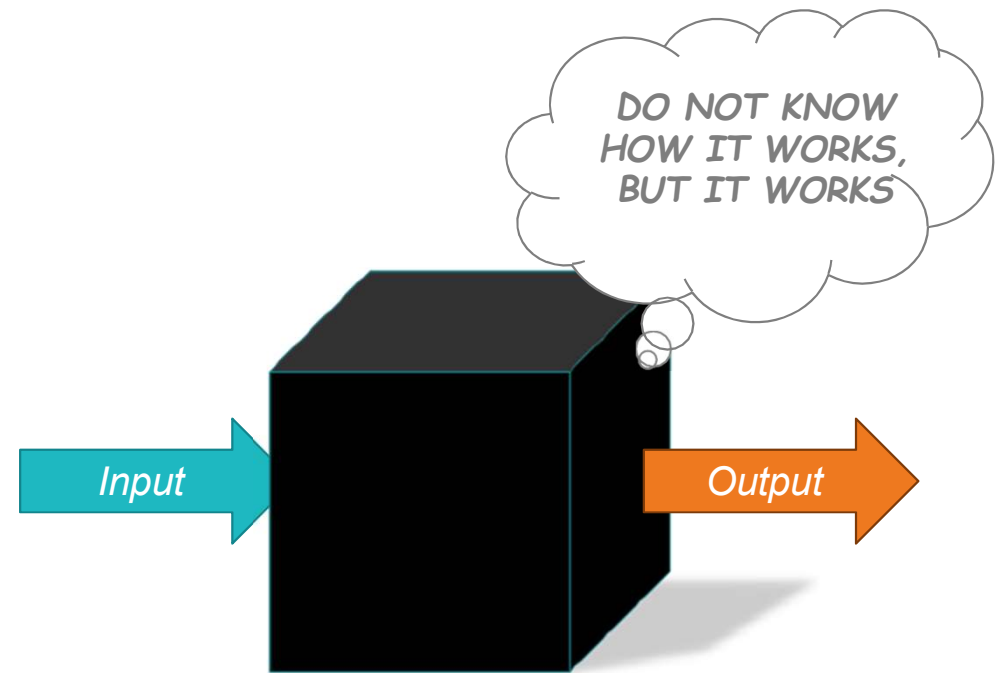
Đóng gói

- Đóng gói (Encapsulation)
 - Dữ liệu và phương thức được đóng gói trong một lớp
 - Dữ liệu được che giấu ở bên trong lớp và chỉ được truy cập và thay đổi qua các phương thức được cung cấp ra bên ngoài



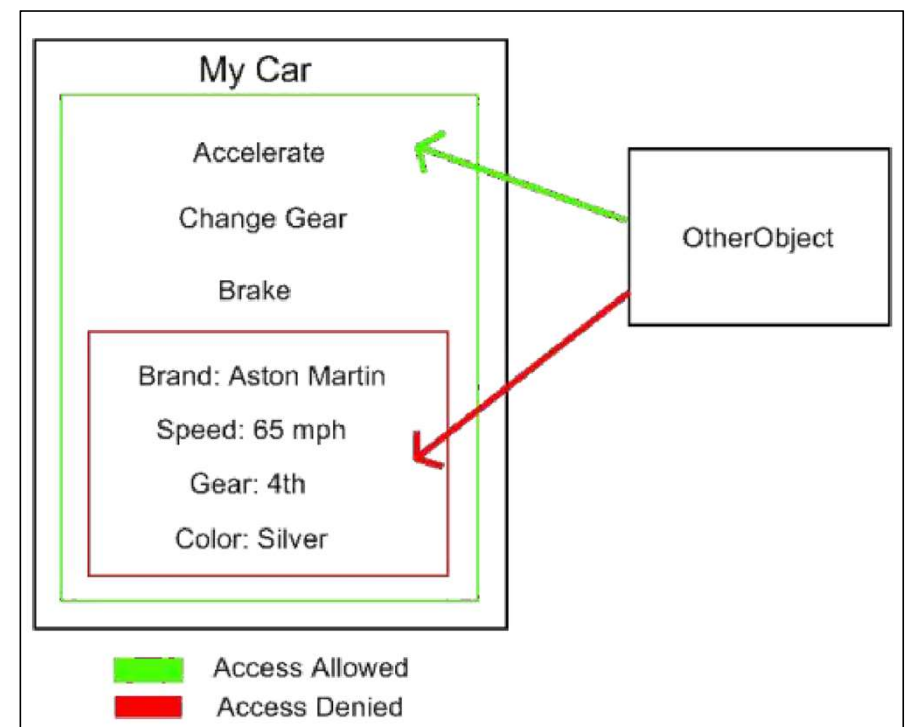
Đóng gói

- Một đối tượng là một thực thể được đóng gói, cung cấp tập các dịch vụ/hành vi nhất định
- Một đối tượng được đóng gói có thể được xem như một hộp đen – các tác vụ thực thi là ẩn so với client/caller



Che giấu dữ liệu

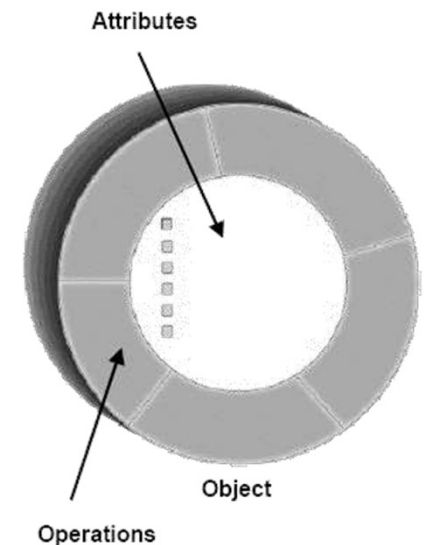
- Dữ liệu được che giấu ở bên trong lớp và chỉ được truy cập và thay đổi qua các Phương thức cung cấp ra bên ngoài
- Tránh thay đổi trái phép hoặc làm sai lệch dữ liệu



Cơ chế che giấu dữ liệu

- Các thành viên dữ liệu:
 - Chỉ có thể truy cập từ các phương thức/lệnh bên trong lớp
 - Chỉ định truy cập là private để bảo vệ dữ liệu (access modifier)
- Các đối tượng khác muốn truy nhập vào dữ liệu riêng tư này phải thông qua các phương thức public

BankAccount
- owner: String - balance: double
+ debit(double): boolean + credit(double)



Cơ chế che giấu dữ liệu

- Các thành phần dữ liệu là private => Để truy cập và chỉnh sửa các giá trị của dữ liệu, lớp cần phải cung cấp các dịch vụ/Phương thức
 - Accessor (getter): Trả về giá trị hiện tại của một thuộc tính (dữ liệu)
 - Mutator (setter): Thay đổi giá trị của một thuộc tính

```
public class Account
{
    private int accID;
    private string accName;
    private double accBalance;

    // Getter and Setter for accID
    public int AccID
    {
        get { return accID; }
        set { accID = value; }
    }
    // ...
}
```

```
package com.megabank.models;

public class BankAccount {
    private String owner;
    private double balance = 0.0;
}
```

```
public String getOwner() {
    return owner;
}
```



Phương thức Get

- Các phương thức để truy vấn (query method, accessor) là các phương thức dùng để đọc giá trị của các thành viên dữ liệu
- Có nhiều loại truy vấn:
 - Truy vấn đơn giản (“giá trị của x là bao nhiêu?”)
 - Truy vấn điều kiện (“thành viên x có lớn hơn 10 không?”)
 - Truy vấn dẫn xuất (“tổng giá trị của các thành viên x và y là bao nhiêu?”)
- Đặc điểm quan trọng của phương thức truy vấn là nó không nên thay đổi trạng thái hiện tại
 - không thay đổi giá trị của thành viên dữ liệu nào

• Các phương thức truy vấn (query method, accessor) là các phương thức dùng để hỏi về giá trị của các thành

Phương thức Set

- Các phương thức thiết lập (Mutator, setter) là các phương thức dùng để thay đổi giá trị các thành viên dữ liệu
- Ưu điểm của việc sử dụng các phương thức setter là có thể sử dụng các phương thức setter để đảm bảo tính hợp lệ của các thành phần dữ liệu
 - Kiểm tra giá trị đầu vào trước khi gán vào các thuộc tính

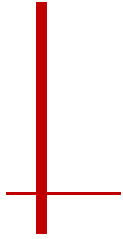
Ví dụ code java

```
public class Time {  
    private int hour;  
    private int minute;  
    private int second;  
  
    public Time () {  
        setTime(0, 0, 0);  
    }  
  
    public void setHour (int h) { hour = ( ( h >= 0 && h < 24 ) ? h : 0 ); }  
  
    public void setMinute (int m) { minute = ( ( m >= 0 && m < 60 ) ? m : 0 ); }  
  
    public void setSecond (int s) { second = ( ( s >= 0 && s < 60 ) ? s : 0 ); }  
  
    public void setTime (int h, int m, int s) {  
        setHour(h);  
        setMinute(m);  
        setSecond(s);  
    }  
  
    public int getHour () { return hour; }  
  
    public int getMinute () { return minute; }  
  
    public int getSecond () { return second; }  
}
```

restricted access: private
members are *not*
externally accessible; but
we need to know and
modify their values

set methods: public
methods that allow
clients to *modify*
private data; also
known as *mutators*

get methods: public
methods that allow
clients to *read private*
data; also known as
accessors





ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

2. Hàm khởi tạo / constructor



Khởi tạo đối tượng

- Làm việc với đối tượng
 - làm việc qua tên (biến đối tượng)
 - Trước khi có thể sử dụng đối tượng => cần khởi tạo đối tượng
 - Vòng đời: khởi tạo => sử dụng => giải phóng
- Khởi tạo: từ khóa **new**

```
BankAccount acc;  
acc = new BankAccount();
```
- Để khởi tạo:
 - Phương thức khởi tạo (constructor) được gọi
 - Có thể lập trình các constructor
 - Nếu không: implicit constructor được gọi, gán giá trị mặc định

Phương thức khởi tạo - constructor

- Là phương thức được gọi để khởi tạo đối tượng => thường là tính toán, gán các giá trị cho các thành phần dữ liệu của đối tượng
- Đặc điểm
 - Tên phương thức - trùng tên lớp
 - Không có giá trị trả về
 - Có thể có nhiều constructor (khác nhau bởi tham số)
 - Constructor không được coi là thành viên của lớp

Phương thức khởi tạo

- VD Constructor mặc định
 - Với C#: khi không viết mã constructor
- VD Constructor không có tham số

```
public class Course
{
    public string CourseCode ;
    public string CourseName ;
    public int Credit ;
}
//Caller
Course course = new Course();
```

```
public class Course
{
    public string CourseCode ;
    public string CourseName ;
    public int Credit = 0;
    public Course()
    {
        CourseCode = "unknown";
        CourseName = "undefined";
    }
}
```

Phương thức khởi tạo

- VD: Phương thức khởi tạo có tham số

```
public class Student
{
    public string StudentCode { get; set; }
    public string FullName { get; set; }
    public DateTime Birthday { get; set; }
    public Student(string studentCode, string fullName, DateTime birthday)
    {
        StudentCode = studentCode;
        FullName = fullName;
        Birthday = birthday;
    }
}

// caller ...
Student sv1 = new Student("PH001", "Nguyễn Văn A", new DateTime(2002, 1, 1));
Console.WriteLine($"Sinh viên {sv1.FullName} học ở {Student.SchoolName} ");
```

Khởi tạo riêng tư private constructor (C#)

- Khi xây dựng lớp đặc biệt, vd lớp tiện ích, và dự định không được tạo đối tượng, không được kế thừa => xây dựng private constructor

```
class MathLib
{
    public static double PI = 3.14;

    private MathLib()
    {
    }
}
```

Khởi tạo tĩnh static constructor (C#)

- Để khởi tạo các thành viên static property
- Phương thức static constructor, không có tham số
- Được tự động thực thi khi sử dụng lớp lần đầu

```
class MyColorCode
{
    public static string color_success;
    public static string color_danger;
    public static string color_warning;

    static MyColorCode()
    {
        Console.WriteLine("Static Contructor
Call");
        color_success = "Green";
        color_danger = "Red";
        color_warning = "Yellow";
    }
}
Console.WriteLine(MyColorCode.color_success);
```

Hàm hủy - destructor

- Hàm thành viên có chức năng ngược lại constructor
- Được gọi/Thực thi vào lúc giải phóng đối tượng
- Thường dùng để thực hiện tác vụ "dọn dẹp": giải phóng tài nguyên đối tượng đang quản lý

```
~TênLớp()  
{  
    // cleanup statements  
}
```

Hàm hủy - destructor

- C#: destructor là hàm thành viên đặc biệt (cũng như constructor)
- Tên giống tên lớp với dấu ~ ở trước

```
~Student()  
{  
    // cleanup statements  
}
```

- Đặc điểm
 - Mỗi lớp chỉ có 1 destructor
 - Không chỉ định phạm vi truy cập
 - Không kế thừa, nạp chồng
- Nếu không viết mã: HT tự tạo hàm destructor ngầm định

Khởi tạo đối tượng trong C++



- Phương thức khởi tạo: Tương tự Java

- Khởi tạo đối tượng

- Đối với đối tượng

```
BankAccount acc;
```

```
BankAccount acc("Tran Van A", 100000);
```

- Đối với con trỏ đối tượng

```
BankAccount *acc = new BankAccount();
```

```
BankAccount *acc = new BankAccount("Tran Van A", 100000);
```

Hàm hủy C++



- Ngược lại với quá trình khởi tạo đối tượng, khi giải phóng đối tượng chúng ta phải giải phóng toàn bộ bộ nhớ đã được cấp phát cho đối tượng. Chức năng của hàm hủy (destructor) sẽ thực hiện vai trò này
- Ví dụ:

```
class A { int n;  
public:  
    A(); // constructor  
    ~A(); // destructor  
};
```

- Java: không dùng hàm hủy.

Ví dụ 1 (java)

```
public class BankAccount {  
    private String owner;  
    private double balance;  
}  
  
public class Test {  
    public static void main(String args[]) {  
        BankAccount acc1 = new BankAccount();  
    }  
}
```

→ Phương thức khởi tạo mặc định do Java cung cấp.

Ví dụ 2 (java)

```
public class BankAccount {  
    private String owner;  
    private double balance;  
    public BankAccount() {  
        owner = "noname";  
    }  
}  
  
public class Test {  
    public static void main(String args[]) {  
        BankAccount acc1 = new BankAccount();  
    }  
}
```

→ Phương thức khởi tạo mặc định tự viết.

Ví dụ 3 (java)

```
public class BankAccount {
    private String owner;
    private double balance;
    public BankAccount(String name) {
        owner = name;
    }
}

public class Test {

    public static void main(String args[]) {
        BankAccount account1 = new
            BankAccount(); //Error
        BankAccount account2 = new
            BankAccount("Hoang");
    }
}
```



Hủy đối tượng - Gabage collector





ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

3. Nạp chồng (Overloading)



Chữ ký phương thức

- Chữ ký - phân biệt Phương thức gồm
 - Tên
 - Số lượng tham số & kiểu

The diagram shows a Java method signature: `public void credit(double amount) {`. The entire signature is enclosed in a brown rectangular box. An arrow labeled "method name" points to the word "credit". An arrow labeled "argument type" points to the word "double". An arrow labeled "signature" points to the entire "credit(double amount)" part. The word "public" is to the left of the box, "void" is to the right of the box, and an opening curly brace "{" is at the end of the line. Below the box, there are three dots "..." and a closing curly brace "}".

```
public void credit(double amount) {  
    ...  
}
```

Nạp chồng

- Nạp chồng hay chồng phương thức (method overloading)
 - Các phương thức trong cùng một lớp có thể trùng tên nhưng khác chữ ký
 - Số lượng tham số khác nhau
 - Nếu cùng số lượng tham số thì kiểu dữ liệu khác nhau
- Mục đích
 - Tên trùng nhau để mô tả bản chất công việc
 - Thuận tiện cho lập trình vì không cần phải nhớ quá nhiều tên phương thức mà chỉ cần nhớ một tên và lựa chọn các tham số cho phù hợp

Ví dụ

```
class MyDate {  
    private int year, month, day;  
    public bool setMonth(int m){  
        if (m < 1 || m > 12)  
            return false;  
        month = m;  
        return true;  
    }  
    public bool setMonth(String s)  
    {  
        int m = int.Parse(s);  
        return setMonth(m);  
    }  
}
```

```
class MyDate {  
    int year, month, day;  
    public boolean setMonth(int m)  
    { ...}  
    public boolean setMonth(String s)  
    { ...}  
}  
  
public class Test{  
    public static void main(args[]){  
        String MyDate d = new MyDate();  
        d.setMonth(9);  
        d.setMonth("9");  
    }  
}
```

Ví dụ về nạp chồng

- Trong C#: `Console.WriteLine` có hơn 10 khai báo nạp chồng

vd:

```
Console.WriteLine()  
Console.WriteLine(string value)  
Console.WriteLine(int value)  
...
```

- Trong Java:



- Phương thức `println()` trong `System.out.println()` có 10 khai báo với các tham số khác nhau: `boolean`, `char[]`, `char`, `double`, `float`, `int`, `long`, `Object`, `String`, và một không có tham số.
- Không cần sử dụng các tên khác nhau (chẳng hạn `printString` hoặc `printDouble`) cho mỗi kiểu dữ liệu muốn hiển thị.

Chú ý

- Các phương thức chỉ được xem xét là chồng khi chúng thuộc cùng một lớp
- Chỉ nên sử dụng kỹ thuật này với các phương thức có cùng mục đích, chức năng; tránh lạm dụng
- Khi dịch, trình dịch căn cứ vào số lượng hoặc kiểu dữ liệu của tham số để quyết định gọi Phương thức nào phù hợp.
- Nếu không chọn được hoặc chọn được nhiều hơn 1 phương thức thì sẽ báo lỗi

Ví dụ

```
void prt(String s) {  
    System.out.println(s); }  
  
void f1(char x) { prt("f1(char)"); }  
void f1(byte x) { prt("f1(byte)"); }  
void f1(short x) { prt("f1(short)"); }  
void f1(int x) { prt("f1(int)"); }  
void f1(long x) { prt("f1(long)"); }  
void f1(float x) { prt("f1(float)"); }  
void f1(double x) { prt("f1(double)"); }
```

- Điều gì xảy ra nếu thực hiện:
 - `f1(5);`
 - `char x='a'; f1(x);`
 - `byte y=0; f1(y);`
 - `float z = 0; f1(z);...`

```
f1(int)  
f1(char)  
f1(byte)  
f1(float)
```

Ví dụ

```
void prt(String s) {  
    System.out.println(s); }  
  
void f3(short x) { prt("f3(short)"); }  
void f3(int x) { prt("f3(int)"); }  
void f3(long x) { prt("f3(long)"); }  
}
```

- Điều gì xảy ra nếu thực hiện:

- `f3(5);`
- `char x='a'; f3(x);`
- `byte y=0; f3(y);`
- `float z = 0; f3(z);`
- `f3(5.5);`

Error: cannot find symbol: method f3(double)

Chồng phương thức khởi tạo (nhắc lại)

- Trong cùng một lớp ta có thể xây dựng nhiều phương thức khởi tạo (constructor) với danh sách tham số khác nhau
=> Chồng phương thức khởi tạo (constructor overloading)
- Nếu muốn gọi đến phương thức khởi tạo khác của lớp: có thể sử dụng toán tử this
`this(danh sách tham số);`



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

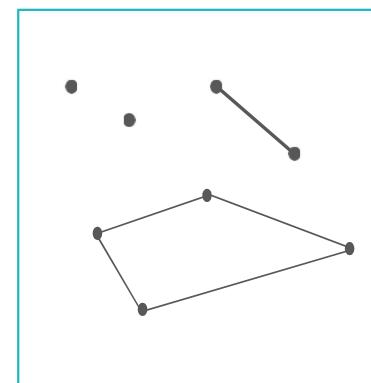
KHOA TOÁN – TIN

4. Kết tập - Aggregation



Mối quan hệ kết tập

- Lớp toàn thể chứa các đối tượng của lớp thành phần
 - ~ Đối tượng lớp thành phần - là một phần của lớp toàn thể
 - Quan hệ chứa "**has a**" hoặc là một phần "**is a part**"
- Ví dụ
 - Ô tô gồm 4 bánh xe
 - Tứ giác gồm 4 điểm



Ví dụ: điểm, tứ giác

```
class Diem{
    public int x, y;
    public Diem(int x, int y){
        this.x = x; this.y = y;
    }
    public void hienThiDiem(){
        Console.WriteLine($"Toa do:{x},{y}");
    }
}
```

Ví dụ: điểm, tứ giác

```
public class TuGiac{
    public Diem A, B, C, D;
    public TuGiac(Diem A, Diem B, Diem C, Diem D){
        this.A = A; this.B = B; this.C = C; this.D = D;
    }
    public void hienThi(){
        Console.WriteLine("Toa do cac dinh cua tu giac:");
        A.hienThiDiem();
        B.hienThiDiem();
        C.hienThiDiem();
        D.hienThiDiem();
    }
}
```

Ví dụ : điểm, tứ giác

```
//caller  
Diem A = new Diem(1, 2);  
Diem B = new Diem(3, 4);  
Diem C = new Diem(5, 6);  
Diem D = new Diem(7, 8);  
TuGiac tg = new TuGiac(A, B, C, D);  
tg.hienThi();
```

Kết tập

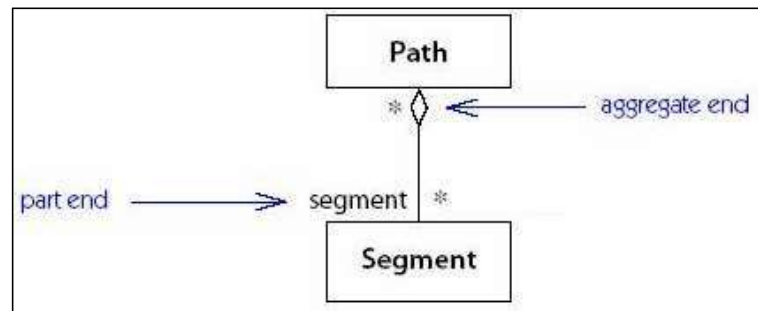
- Kết tập/tổ hợp (aggregate):
 - Các thành phần của lớp mới (toàn thể) là các đối tượng của các lớp có sẵn (thành phần)
 - Tái sử dụng các thành phần dữ liệu và các hành vi của lớp thành phần thông qua đối tượng của lớp thành phần
- Lớp toàn thể (Aggregate/Whole)
- Lớp thành phần (Part).

Thứ tự khởi tạo

- Khi một đối tượng được tạo mới, các thuộc tính của đối tượng đó đều phải được khởi tạo và gán những giá trị tương ứng.
- Các đối tượng thành phần được khởi tạo trước => Các phương thức khởi tạo của các lớp đối tượng thành phần được thực hiện trước

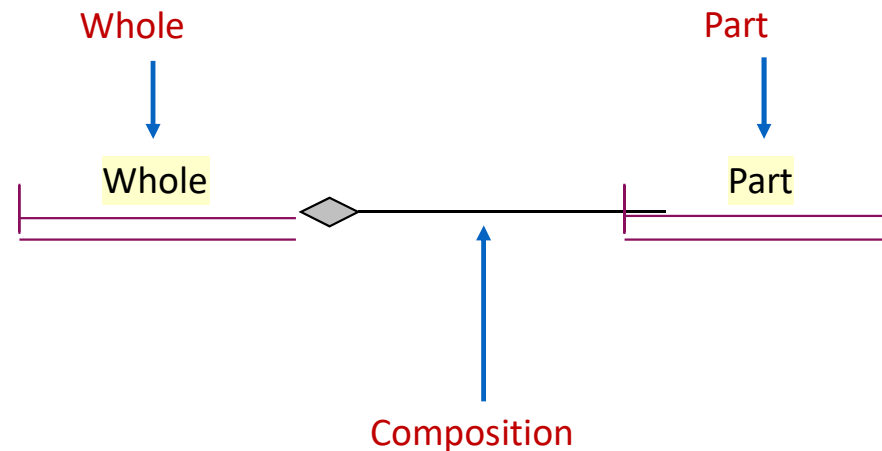
Biểu diễn kết tập bằng UML

- Thể hiện quan hệ
 - các Lớp đối tượng
 - Đường kết nối: hình thoi
 - Số lượng tham gia
 - một số: 1, n
 - một dải: 0..n, 1..n
 - số lượng bất kỳ: *
 - trống: 1



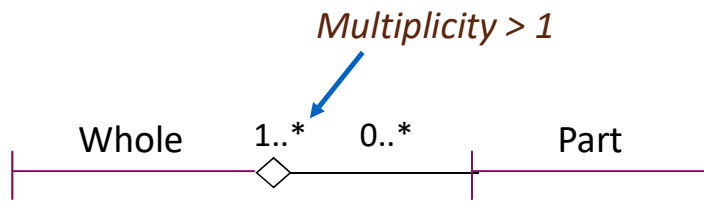
Hợp thành (Composition) là gì?

- Là một dạng của tổ hợp (aggregation), có quan hệ mạnh và cùng vòng đời
 - (obj) Bộ phận không thể tồn tại khi thiếu (obj) tổng thể

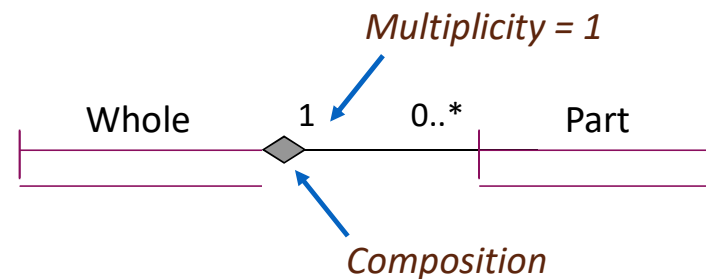
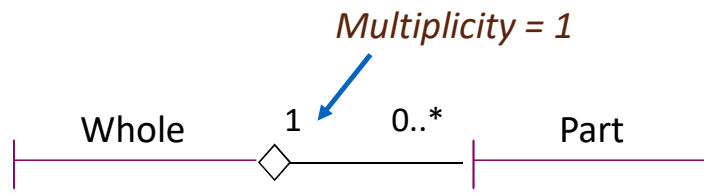


Kết tập/tổ hợp: Chia sẻ hay không chia sẻ

- Shared Aggregation



- Non-shared Aggregation



By definition, composition is non-shared aggregation

Class Diagram Elements

